

OGC2026

2026/09/18

tishii24

Outline

Solution overview

Key findings, overall architecture, and evaluations

Key outcomes and insights

Structuring our approach to integrated packing and scheduling problems

Practical implications

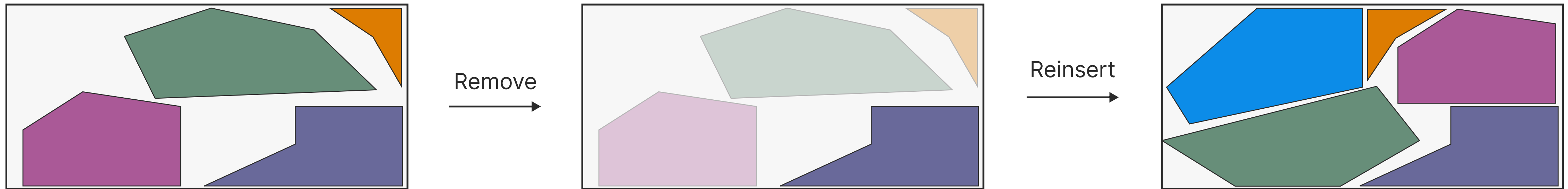
Potential practical benefits beyond solution accuracy

Future directions

Future directions that we were unable to explore during the competition period

Key Findings

- This problem can be viewed as a packing problem in 3D space (x, y, t).
 - ex) A block with processing_time=10 will have length 10 in t-axis.
- For packing problems, **Large Neighborhood Search** is a general heuristic approach.



Typical large neighborhood in packing problems is to remove several blocks and attempt to reinsert them in different positions.

Challenges of applying large neighborhood search to this problem (especially due to its scheduling component):

1. It is difficult to move early blocks in the schedule, since moving early block affects many subsequent blocks.
 - It becomes difficult to transition from a dense packing to another structurally different dense packing.
2. Local search in the direction of simply reducing the objective does not always lead to an overall efficient structure.
 - We need to guide the search towards an efficient structure (explained more later).

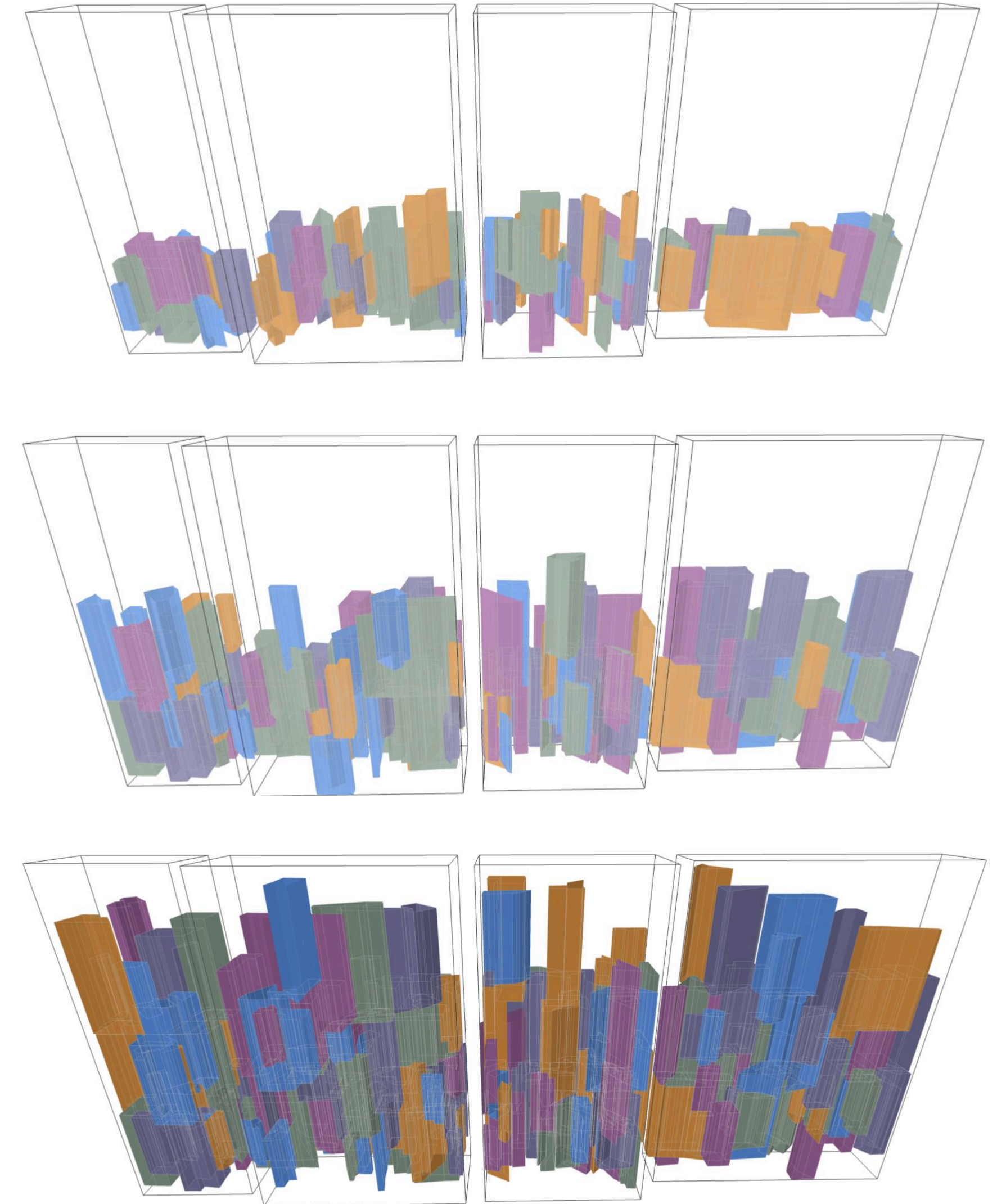
Key Findings

To mitigate the difficulty of moving early blocks, **rolling horizon (repeatedly extends and optimizes a partial problem in several stages)** was effective.

- Since every block is placed at the end of a subproblem at least once, **each block can be tested with relatively few subsequent blocks.**
- This is possible when optimizing each subproblem within a horizon is aligned with optimizing the overall problem.
 - In this problem, dominant objective, tardiness and preference score, was independent for each block, so it was effective.

$$Z_1 = \sum_i T_i, \quad Z_3 = \sum_i (S_i^{\max} - S_{i,b_i}).$$

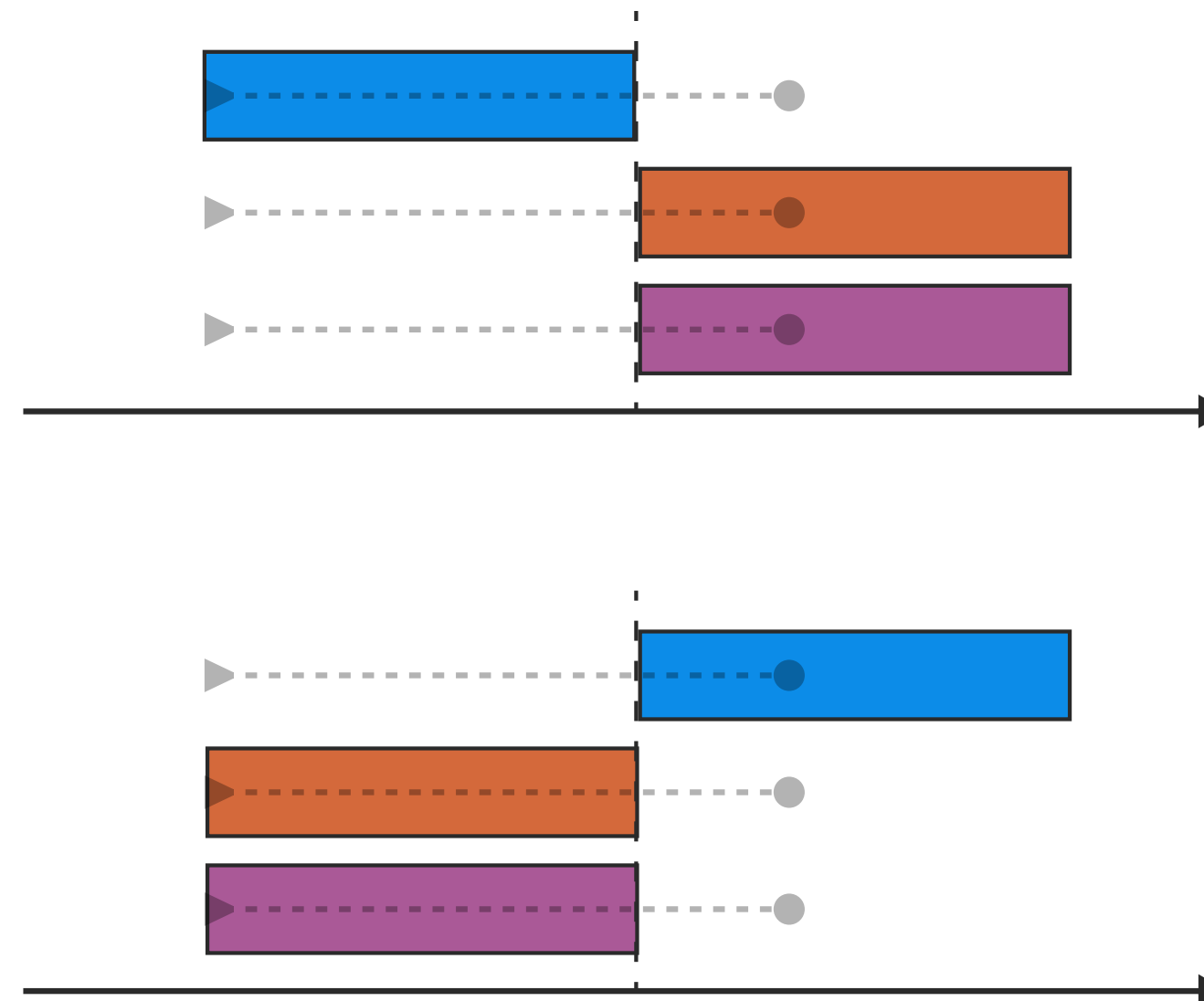
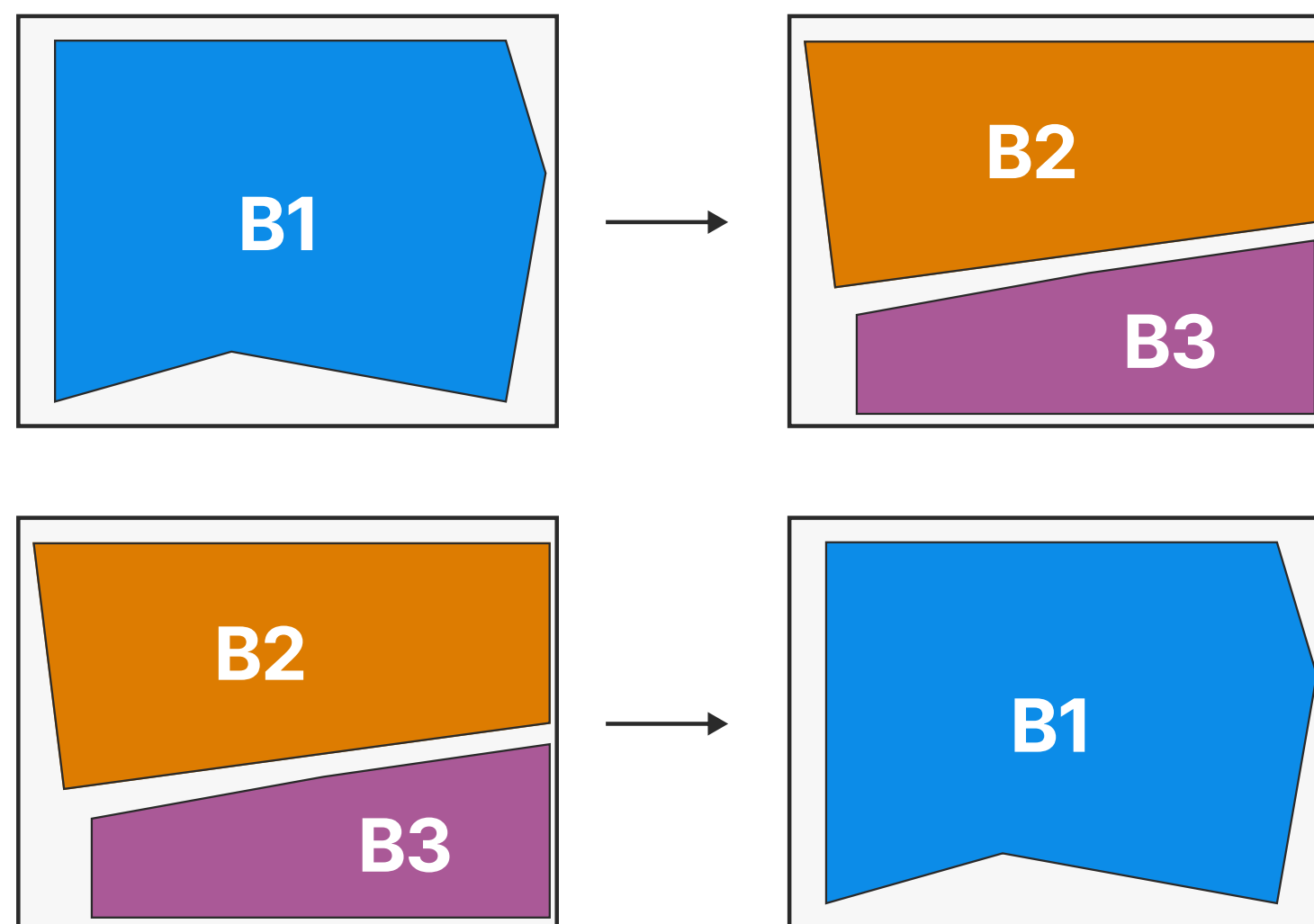
However, to apply rolling horizon, we first need to determine **the order in which blocks are added.**



Key Findings

I found that the addition order need to take care of the following aspects jointly:

- **Scheduling:**
 - Prioritize urgent blocks (e.g. smaller *due_date*).
- **Packability:**
 - Prioritize larger blocks, since smaller blocks have greater flexibility in terms of placement.
- **Objective Efficiency:**
 - It is sometimes preferable to prioritize smaller blocks over larger ones (and this conflicts with packability!)



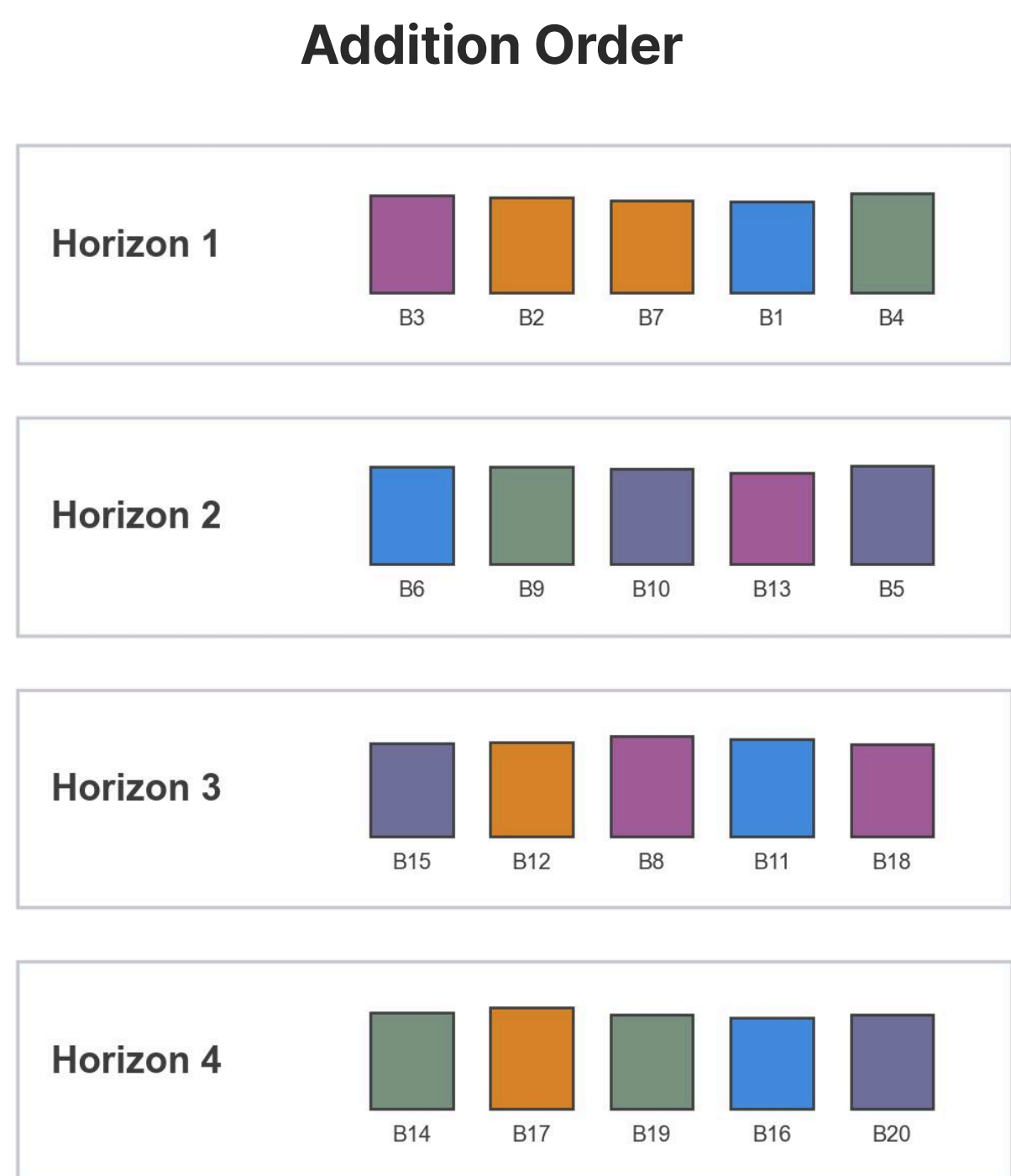
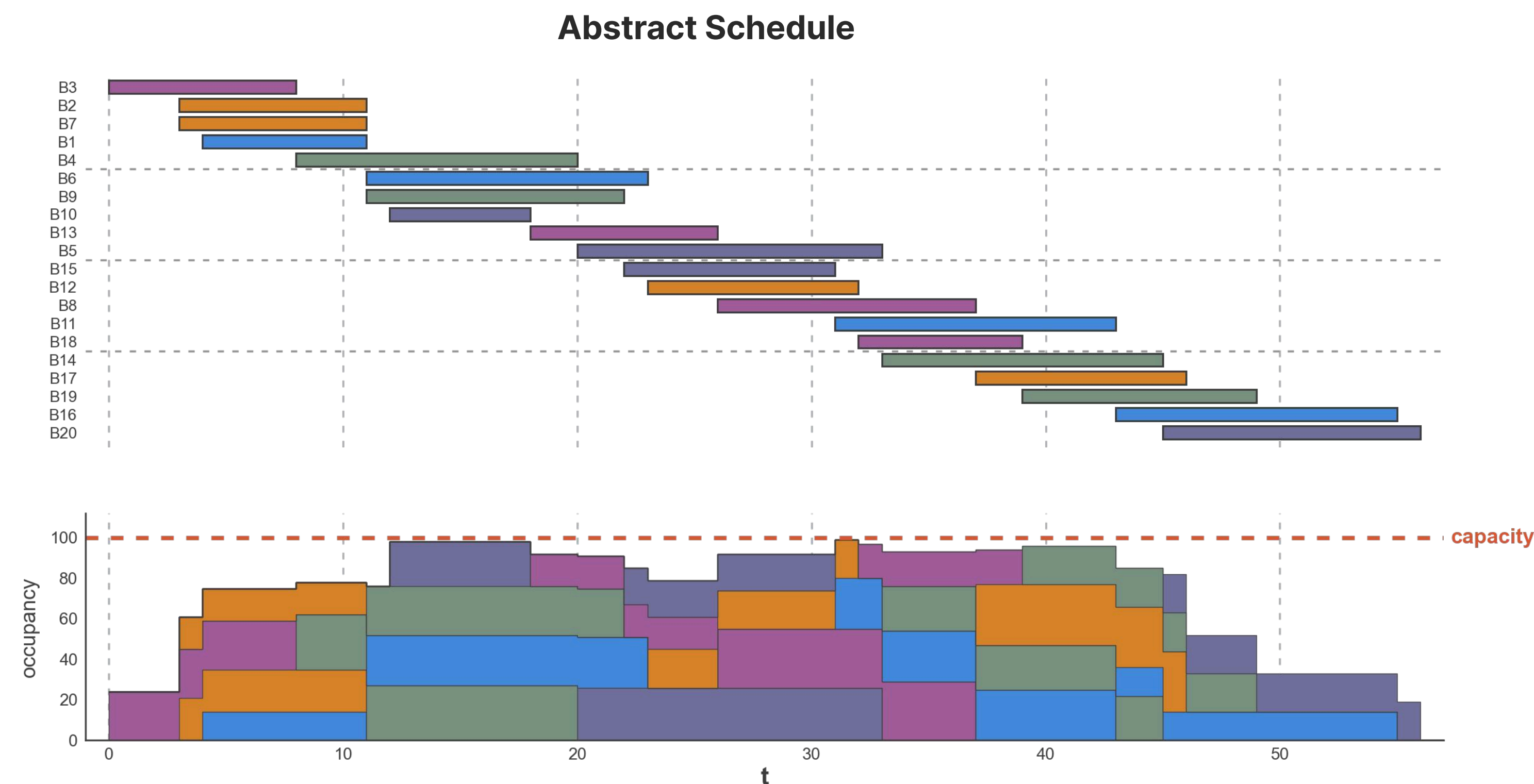
Assuming blocks (B1, B2, B3) with same release/ due/processing time, but the area is $B1 > B2 + B3$.

Prioritizing (B2, B3) than B1 will achieve smaller tardiness compared by prioritizing B1.

Key Findings

I addressed this issue by **pre-solving a relaxed problem that ignores packing (and solve only as a scheduling problem)**.

- The actual placement is not considered; **only the entry time and bay assignment are optimized as variables.**
 - Also, constraints are imposed so that the capacity of each bay is not exceeded at any time.
- Since the problem complexity is significantly reduced, it is easier to find an efficient order.
- **The preoptimized entry time will be used to determine the block addition order for rolling horizon.**
- **Using this addition order will guide the search toward an appropriate solution structure.**

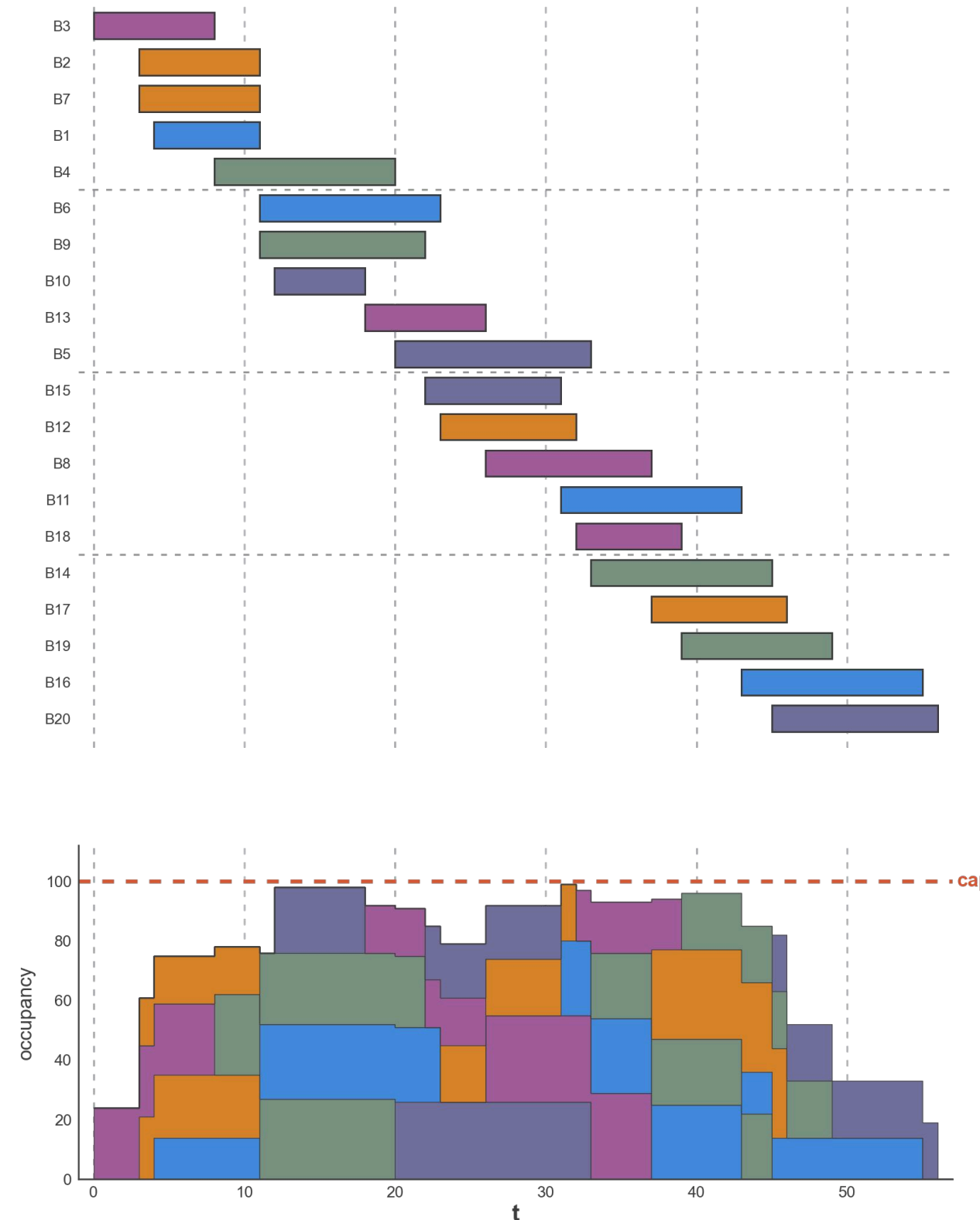


Overall Architecture

Solution: **Preoptimized** the block order and used **rolling horizon** optimization based on **large neighborhood search**.

1st stage

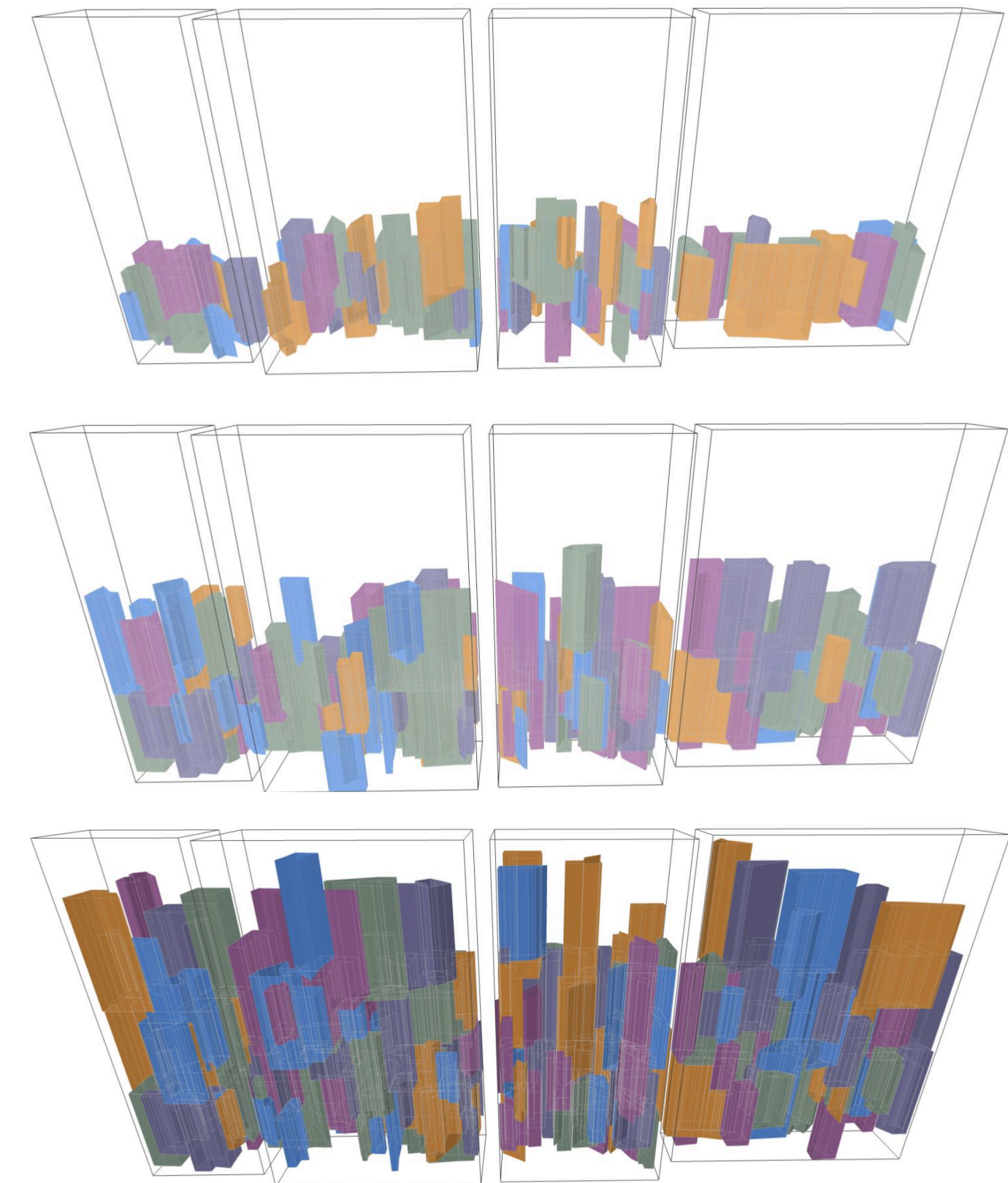
Preoptimize block order ignoring block placement.



block order

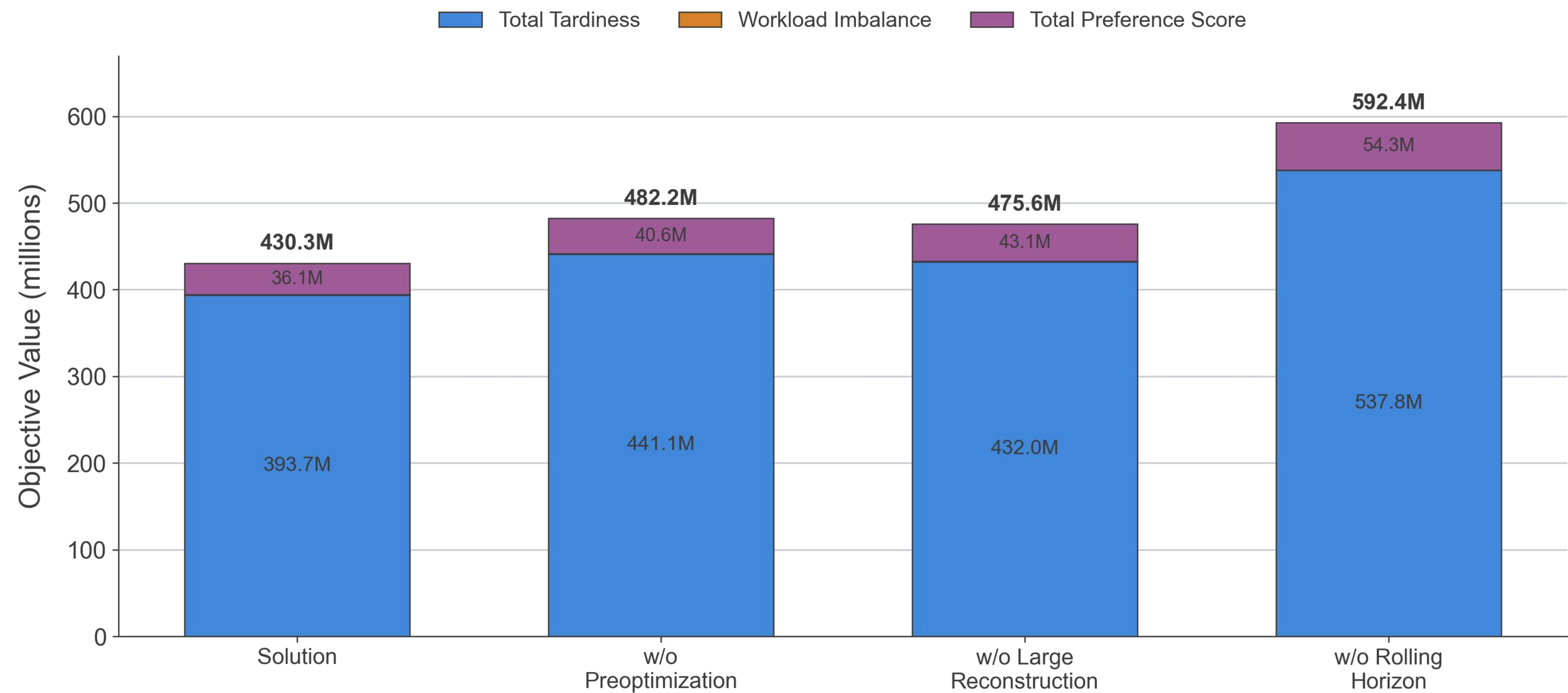
2nd stage

Optimize with rolling horizon, using large neighborhood search.



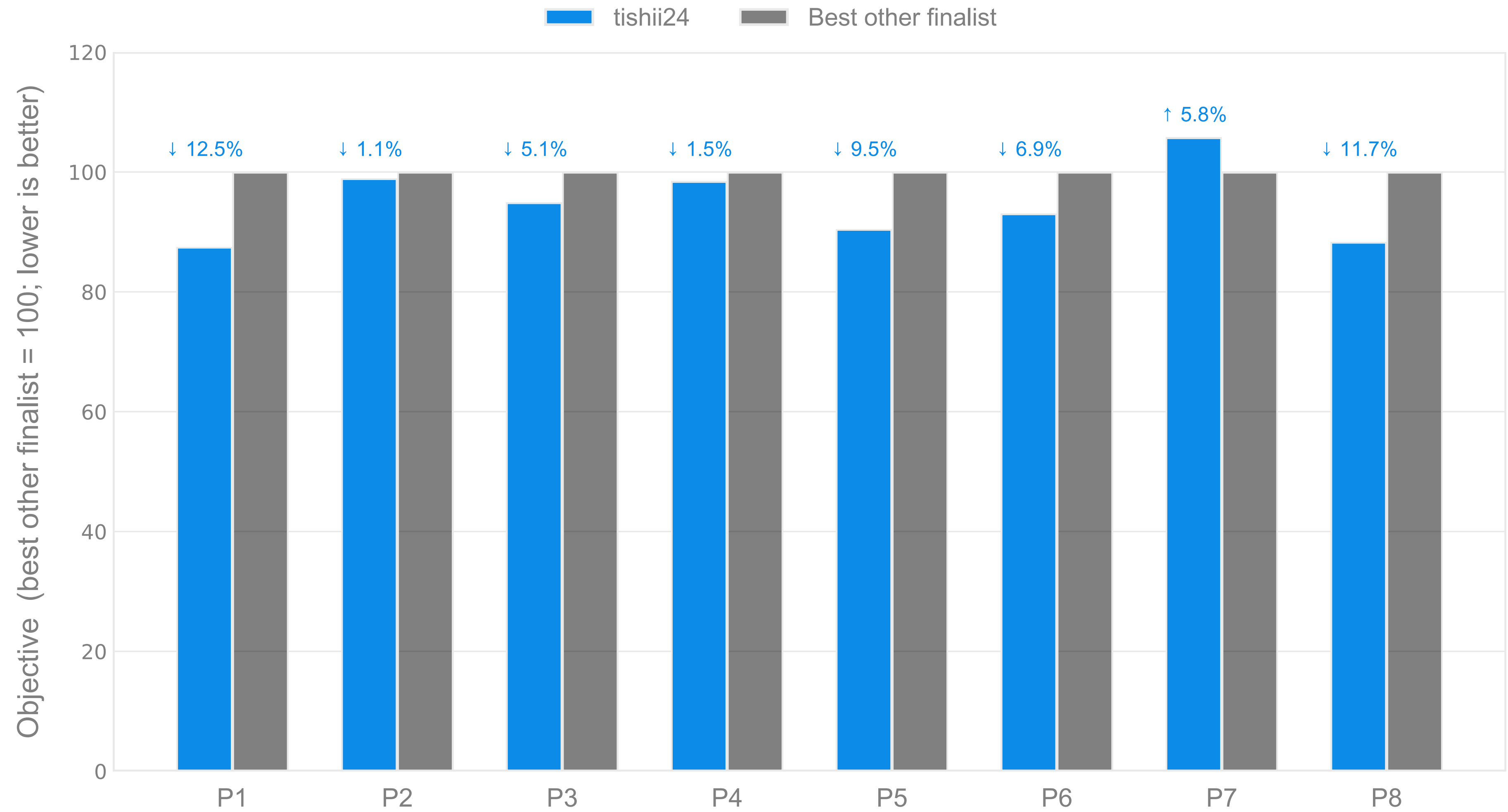
Ablation studies

All ablations increased the total objective more than 10%.



Final Stage Results

Achieved the best score on 7/8 problems. On average across all cases, achieved a **5.3%** lower score.



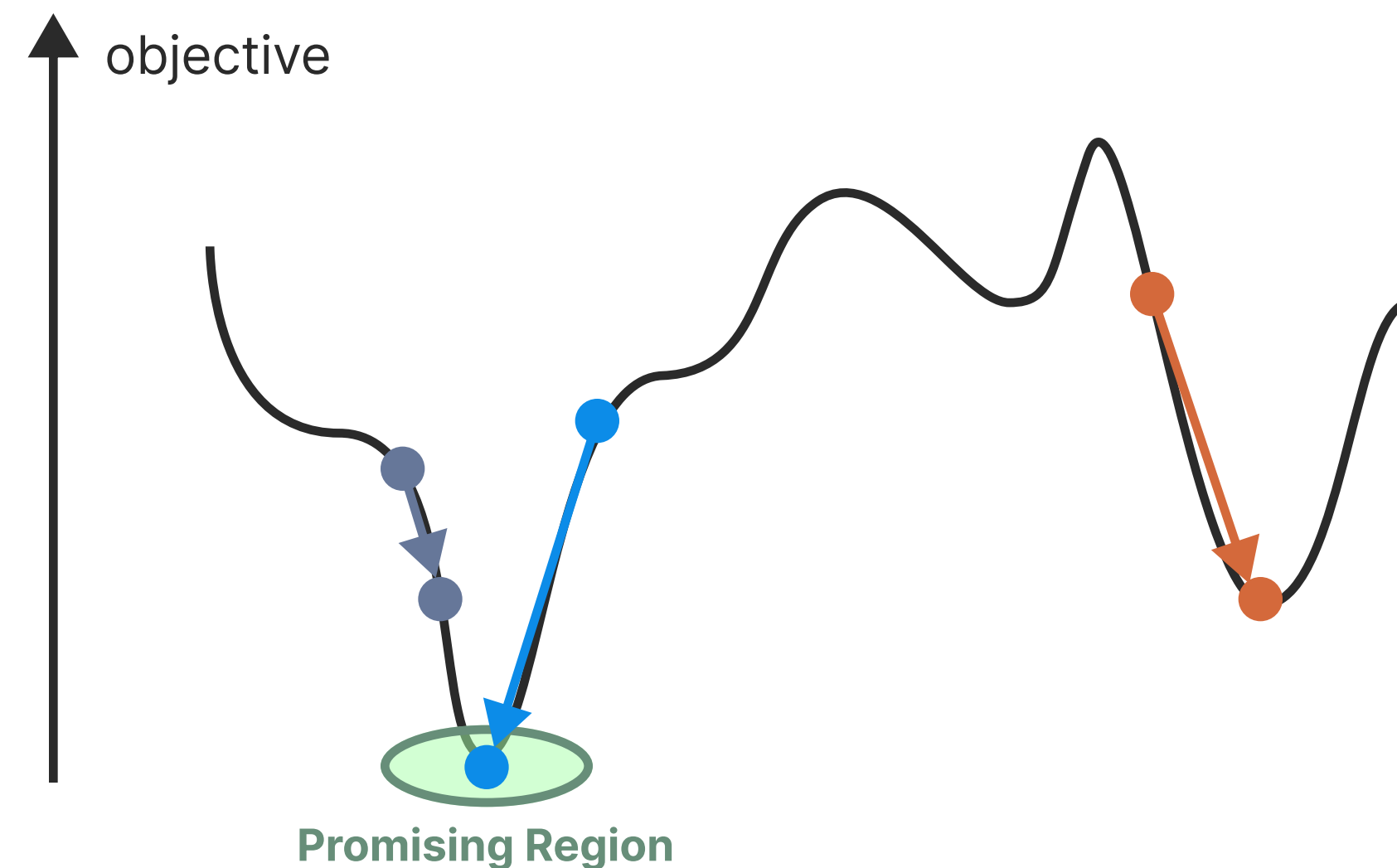
Key Outcomes and Insights

The main challenges of this problem were:

- Strong geometric and temporal constraints make it difficult to move from one local optimum to another.
- An efficient schedule needs to balance many aspects, making it difficult to discover.

To address these challenges, I adopted:

- **Rolling horizon:** mitigate geometric and temporal constraints.
- **Abstract preoptimization:** identify promising regions of the solution space.
- **Using the block order to guide the rolling horizon:** focus the search on these promising regions.



- is w/o rolling-horizon. It has insufficient ability to reach the local optima.
- is w/o preoptimization. It has sufficient ability to reach the local optima, but it cannot find the promising regions itself.
- is the full solution. Guided by preoptimization, it starts near the promising regions, and has sufficient ability to reach the promising optima.

Practical Implications

In practice, there are several aspects that are important as same as solution accuracy.



Accuracy



Efficiency



Scalability



Robustness



Flexibility

I think I proved accuracy and efficiency through final stage results :)

Scalability

Rolling horizon limits the number of blocks optimized at each stage, avoiding a single full-scale optimization.

→ Can handle large problems by dividing the blocks into more horizons.

Robustness

Previously placed blocks remain movable, and large-neighborhood can revise several interdependent decisions together.

→ Can recover from imperfect admission orders or early scheduling decisions.

Flexibility

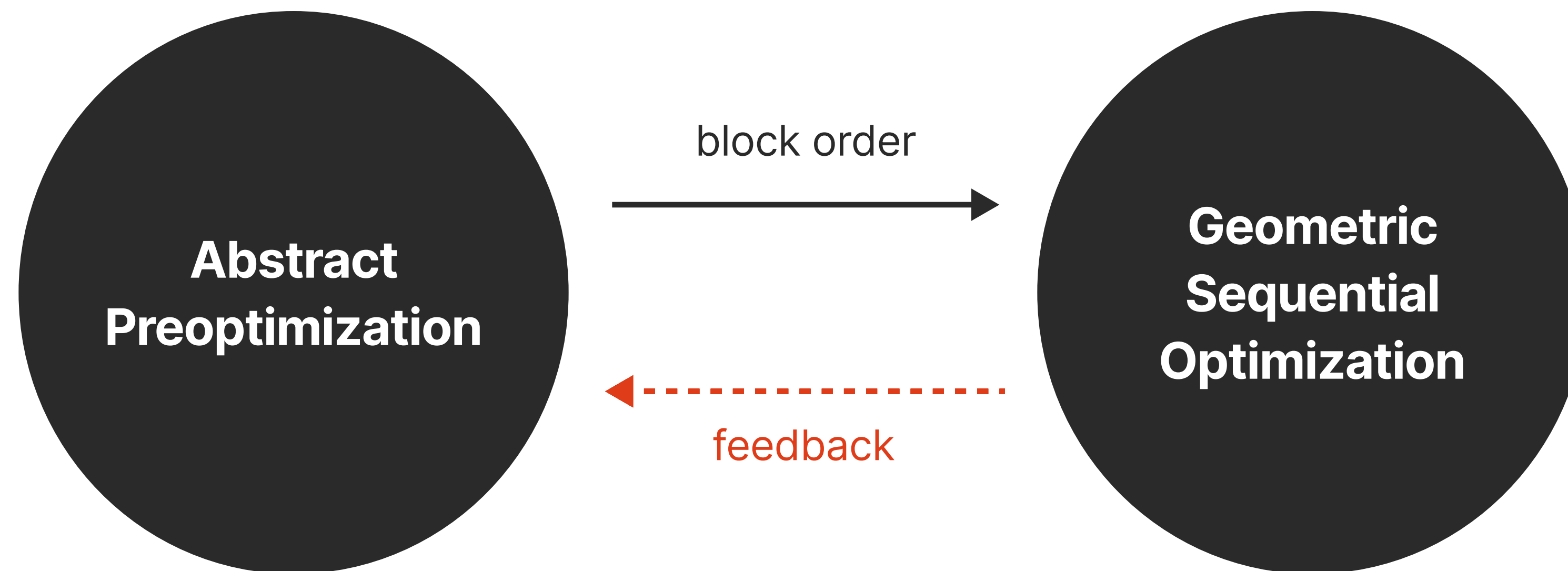
Global guidance, collision detection, and solution improvement are handled by separate components.

→ New constraints or objectives can be introduced into the relevant component without redesigning the entire framework.

Future Directions

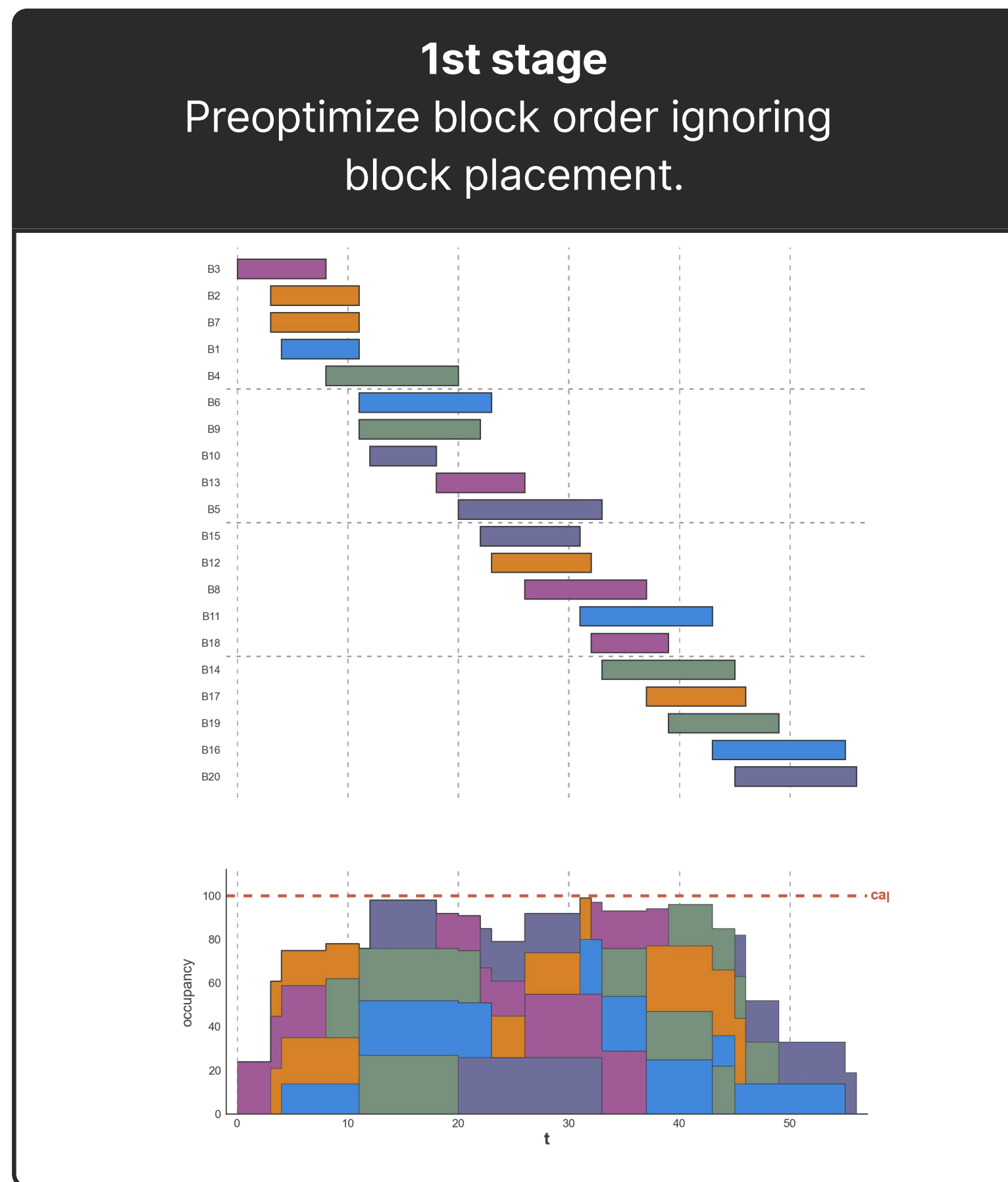
Feedback-Guided Preoptimization

- The current method fixes the block addition order obtained from preoptimization and then performs sequential optimization based on that order.
- The resulting order may be suboptimal when they do not accurately reflect the actual packing and scheduling constraints.
- A promising direction is to use feedback from exact optimization, allowing the preoptimization model to better capture the feasibility in the actual problem.

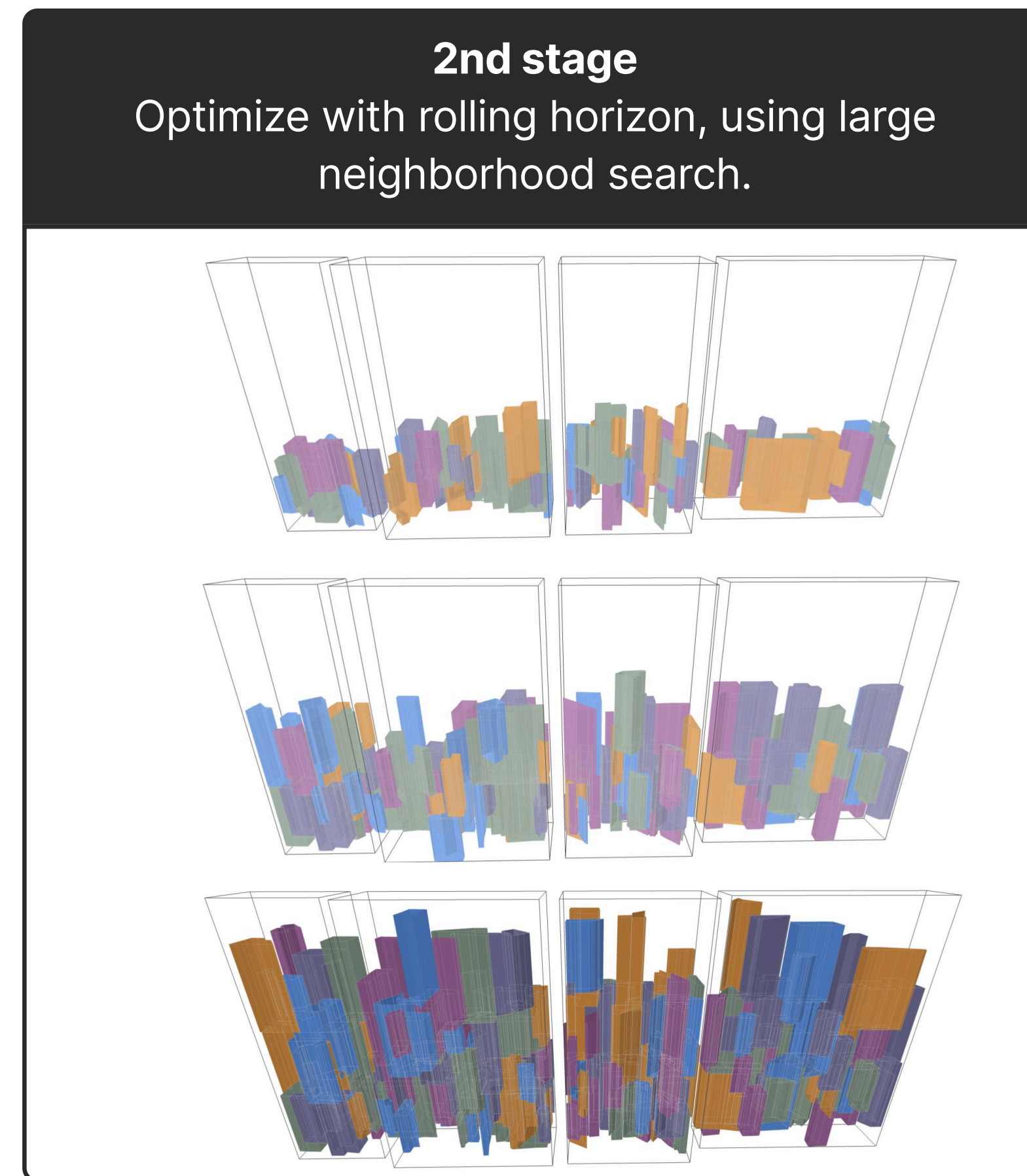


Conclusion

- Introduced a general method to an optimization problem involving both scheduling and packing.
- The solution achieved a score 5% lower than the best score of any other competitor in the final stage.
- Designed the solution to guarantee scalability, robustness, flexibility.



block order
→



감사합니다 :)

Appendix

Overall Algorithm Framework

Algorithm 1 Overall solution procedure

Require: problem instance, horizon size H , global deadline τ

Ensure: feasible geometric schedule S

```

1:  $\{(\hat{b}_i, \hat{e}_i)\}_i \leftarrow \text{ABSTRACTPREOPTIMIZATION}$ 
2:  $\pi \leftarrow$  blocks sorted lexicographically by  $(\hat{e}_i, -V_i, i)$ 
3:  $\mathcal{H} \leftarrow \text{EVENLYPARTITION}(\pi, \lceil N/H \rceil)$ 
4:  $\{S_r\}_r \leftarrow \{\emptyset\}_r$ 
5: for each horizon  $h \in \mathcal{H}$  do
6:   for each worker  $r$  in parallel do
7:      $S_r \leftarrow \text{EXTENDSCHEDULE}(S_r, h)$ 
8:   end for
9:   if  $h$  is the final horizon then
10:     $w'_2 \leftarrow w_2$ 
11:   else
12:     $w'_2 \leftarrow 0$ 
13:   end if
14:    $\tau_h \leftarrow \text{ALLOCATEREMAININGTIME}(h, \mathcal{H}, \tau)$ 
15:    $\{S_r\}_r \leftarrow \text{PARALLELSA}(\{S_r\}_r, w_1, w'_2, w_3, \tau_h)$ 
16: end for
17: return the best  $S_r$ 

```

Abstract preoptimization represents each block i using only:

- Bay assignment $\hat{b}_i$
- Entry time $\hat{e}_i$

The block i approximate occupancy at bay j is calculated as:

$$a_{ij} = \min_o A_{io}^{\text{union}} + \alpha (A_{io}^{\text{bbox}} - A_{io}^{\text{union}}) + \beta L_{io}.$$

where:

- A_{io}^{union} : union area of all layers
- A_{io}^{bbox} : bounding-box area
- L_{io} : perimeter of the union

α and β adjust the occupancy estimate to reflect how difficult each shape is to pack.

At every time t , the total approximate occupancy in each bay must satisfy:

$$\sum_{i: \hat{e}_i \leq t < \hat{e}_i + P_i, \hat{b}_i = j} a_{ij} \leq C_j$$

where C_j is the area of bay j .

Overall Algorithm Framework

Algorithm 1 Overall solution procedure

Require: problem instance, horizon size H , global deadline τ

Ensure: feasible geometric schedule S

```
1:  $\{(\hat{b}_i, \hat{e}_i)\}_i \leftarrow \text{ABSTRACTPREOPTIMIZATION}$ 
2:  $\pi \leftarrow$  blocks sorted lexicographically by  $(\hat{e}_i, -V_i, i)$ 
3:  $\mathcal{H} \leftarrow \text{EVENLYPARTITION}(\pi, \lceil N/H \rceil)$ 
4:  $\{S_r\}_r \leftarrow \{\emptyset\}_r$ 
5: for each horizon  $h \in \mathcal{H}$  do
6:   for each worker  $r$  in parallel do
7:      $S_r \leftarrow \text{EXTENDSCHEDULE}(S_r, h)$ 
8:   end for
9:   if  $h$  is the final horizon then
10:     $w'_2 \leftarrow w_2$ 
11:   else
12:     $w'_2 \leftarrow 0$ 
13:   end if
14:    $\tau_h \leftarrow \text{ALLOCATEREMAININGTIME}(h, \mathcal{H}, \tau)$ 
15:    $\{S_r\}_r \leftarrow \text{PARALLELSA}(\{S_r\}_r, w_1, w'_2, w_3, \tau_h)$ 
16: end for
17: return the best  $S_r$ 
```

Line 2:

Blocks are sorted lexicographically by:

1. Earlier abstract entry time
 2. Larger volume in 3D-space (= area x processing-time)
 3. Smaller block ID (for tie-break)
-

Line 3:

The sorted sequence is evenly partitioned into N/H horizons.

Line 4:

Initialize each worker r schedule S_r as an empty schedule.

Overall Algorithm Framework

Algorithm 1 Overall solution procedure

Require: problem instance, horizon size H , global deadline τ

Ensure: feasible geometric schedule S

```
1:  $\{(\hat{b}_i, \hat{e}_i)\}_i \leftarrow \text{ABSTRACTPREOPTIMIZATION}$ 
2:  $\pi \leftarrow$  blocks sorted lexicographically by  $(\hat{e}_i, -V_i, i)$ 
3:  $\mathcal{H} \leftarrow \text{EVENLYPARTITION}(\pi, \lceil N/H \rceil)$ 
4:  $\{S_r\}_r \leftarrow \{\emptyset\}_r$ 
5: for each horizon  $h \in \mathcal{H}$  do
6:   for each worker  $r$  in parallel do
7:      $S_r \leftarrow \text{EXTENDSCHEDULE}(S_r, h)$ 
8:   end for
9:   if  $h$  is the final horizon then
10:     $w'_2 \leftarrow w_2$ 
11:   else
12:     $w'_2 \leftarrow 0$ 
13:   end if
14:    $\tau_h \leftarrow \text{ALLOCATEREMAININGTIME}(h, \mathcal{H}, \tau)$ 
15:    $\{S_r\}_r \leftarrow \text{PARALLELSA}(\{S_r\}_r, w_1, w'_2, w_3, \tau_h)$ 
16: end for
17: return the best  $S_r$ 
```

Line 6-8:

At each horizon, every worker independently extends its current schedule using a similar procedure of large reconstruct neighborhood (explained later).

Line 9-13:

Workload imbalance is evaluated only in the final horizon.

Line 14:

The time allocated to horizon h is proportional to:

$$\tau_h = \tau_{\text{total}} \frac{N_h^\gamma}{\sum_q N_q^\gamma}.$$

where N_h is the number of admitted blocks in horizon h .

Overall Algorithm Framework

Algorithm 1 Overall solution procedure

Require: problem instance, horizon size H , global deadline τ

Ensure: feasible geometric schedule S

```
1:  $\{(\hat{b}_i, \hat{e}_i)\}_i \leftarrow \text{ABSTRACTPREOPTIMIZATION}$ 
2:  $\pi \leftarrow$  blocks sorted lexicographically by  $(\hat{e}_i, -V_i, i)$ 
3:  $\mathcal{H} \leftarrow \text{EVENLYPARTITION}(\pi, \lceil N/H \rceil)$ 
4:  $\{S_r\}_r \leftarrow \{\emptyset\}_r$ 
5: for each horizon  $h \in \mathcal{H}$  do
6:   for each worker  $r$  in parallel do
7:      $S_r \leftarrow \text{EXTENDSCHEDULE}(S_r, h)$ 
8:   end for
9:   if  $h$  is the final horizon then
10:     $w'_2 \leftarrow w_2$ 
11:   else
12:     $w'_2 \leftarrow 0$ 
13:   end if
14:    $\tau_h \leftarrow \text{ALLOCATEREMAININGTIME}(h, \mathcal{H}, \tau)$ 
15:    $\{S_r\}_r \leftarrow \text{PARALLELSA}(\{S_r\}_r, w_1, w'_2, w_3, \tau_h)$ 
16: end for
17: return the best  $S_r$ 
```

Local Neighborhood Search by Parallel Simulated Annealing

- Each worker independently maintains and improves its own schedule.
- Workers update a shared global-best solution and periodically import it when it is sufficiently better.

Four neighborhoods are used:

- Shift: locally changes the position and entry time
- Move: removes and greedily reinserts one block
- Rotate: changes the orientation and searches a new placement
- **Large Reconstruction: removes and reinserts multiple blocks**

Small neighborhoods are used to change the state in low cost.

Overall Algorithm Framework

Algorithm 1 Overall solution procedure

Require: problem instance, horizon size H , global deadline τ

Ensure: feasible geometric schedule S

```
1:  $\{(\hat{b}_i, \hat{e}_i)\}_i \leftarrow \text{ABSTRACTPREOPTIMIZATION}$ 
2:  $\pi \leftarrow$  blocks sorted lexicographically by  $(\hat{e}_i, -V_i, i)$ 
3:  $\mathcal{H} \leftarrow \text{EVENLYPARTITION}(\pi, \lceil N/H \rceil)$ 
4:  $\{S_r\}_r \leftarrow \{\emptyset\}_r$ 
5: for each horizon  $h \in \mathcal{H}$  do
6:   for each worker  $r$  in parallel do
7:      $S_r \leftarrow \text{EXTENDSCHEDULE}(S_r, h)$ 
8:   end for
9:   if  $h$  is the final horizon then
10:     $w'_2 \leftarrow w_2$ 
11:   else
12:     $w'_2 \leftarrow 0$ 
13:   end if
14:    $\tau_h \leftarrow \text{ALLOCATEREMAININGTIME}(h, \mathcal{H}, \tau)$ 
15:    $\{S_r\}_r \leftarrow \text{PARALLELSA}(\{S_r\}_r, w_1, w'_2, w_3, \tau_h)$ 
16: end for
17: return the best  $S_r$ 
```

Large Reconstruction

1. Randomly select multiple seed blocks.
2. For each seed s , calculate the distance in 3D space to all other blocks in the same bay:

$$d(i, s) = \lambda_x |x_i - x_s|^p + \lambda_y |y_i - y_s|^p + \lambda_t |e_i - e_s|^p, \quad p \sim U[2, 4],$$

The distance weights $\lambda_x, \lambda_y, \lambda_t$ and exponent p are sampled for each reconstruction for randomness.

3. Remove several blocks with small $d(i, s)$.
4. Reinsert the removed blocks in a randomized order using greedy insertion. Insertion is performed by minimizing the score delta:

$$\Delta F(s) = w_1 T_i(s) + w_2 \Delta Z_2(s) + w_3 (S_i^{\max} - S_{i, b_i(s)}).$$

Among candidates tied, an earlier entry time and placement near a bay corner are preferred.

...after applying these procedures for each horizon, the solution returns the global best schedule.

Key Ideas that Failed

Beam-Search Reconstruction

Reusing insertion results was less important than exploring diverse placements and did not improve the solution quality.

Multi-Stage Optimization

Optimizing tardiness, preference, and workload separately showed no clear advantage over optimizing them jointly.

Blocker-Aware Removal Methods

Explicitly removing blockers in the schedule to reduce penalty were promising, but difficult to tune consistently.

Further Implementation Plan

Improve the Relaxed Model

Refine the approximate occupancy to reduce the gap between abstract and geometric schedules.

Adaptive Rolling Horizons

Dynamically determine the horizon size and time allocation according to instance size and congestion.

Worker Specialization

Assign different temperature ranges or neighborhood compositions to workers for structural and local improvements.

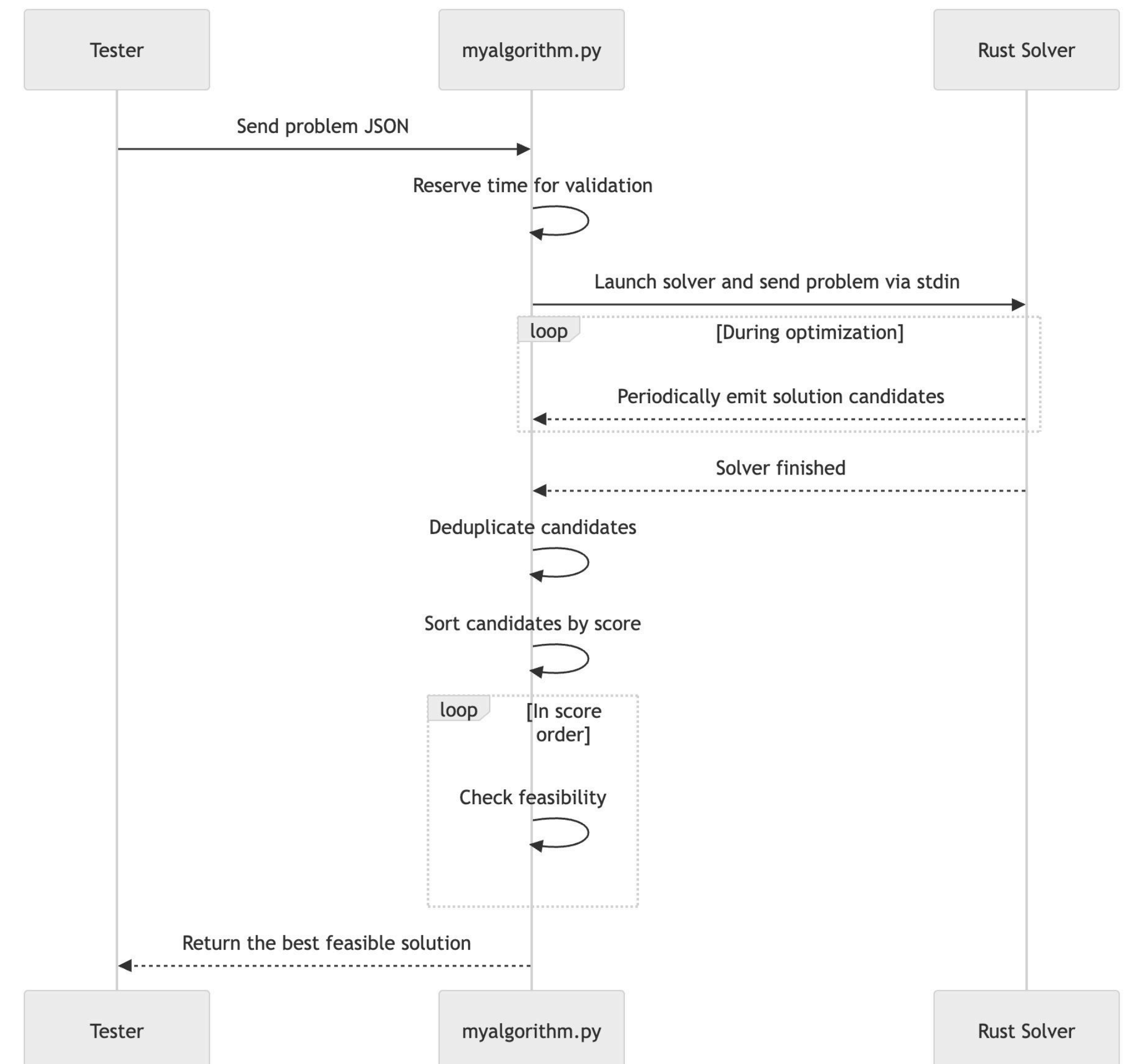
Adaptive Temperature Control

Adjust the temperature for each instance and horizon based on observed score differences and acceptance rates.

Implementation Details

Rust performs the computationally intensive optimization, while Python wrapper (`myalgorithm.py`) manages candidate selection and final feasibility validation.

- The optimization solver was implemented in Rust.
- Instead of returning only the final state, the Rust solver periodically emits improving solution candidates to the `myalgorithm.py` during optimization.
- `myalgorithm.py` manages the time limit, collects and deduplicates candidates, sorts them by score, and validates feasibility before returning the best feasible solution.



I added this mechanism as a safeguard against unexpected failures on hidden test cases. Apparently, the same inputs were used, so I think it did not activate :)

Computation Results

Experimental Setup

- Instances: 40 instances from the final training dataset
- Time Limit: 300 seconds per instance
- Execution Environment: Apple M1 Pro with 8 CPU cores and 32 GB of memory

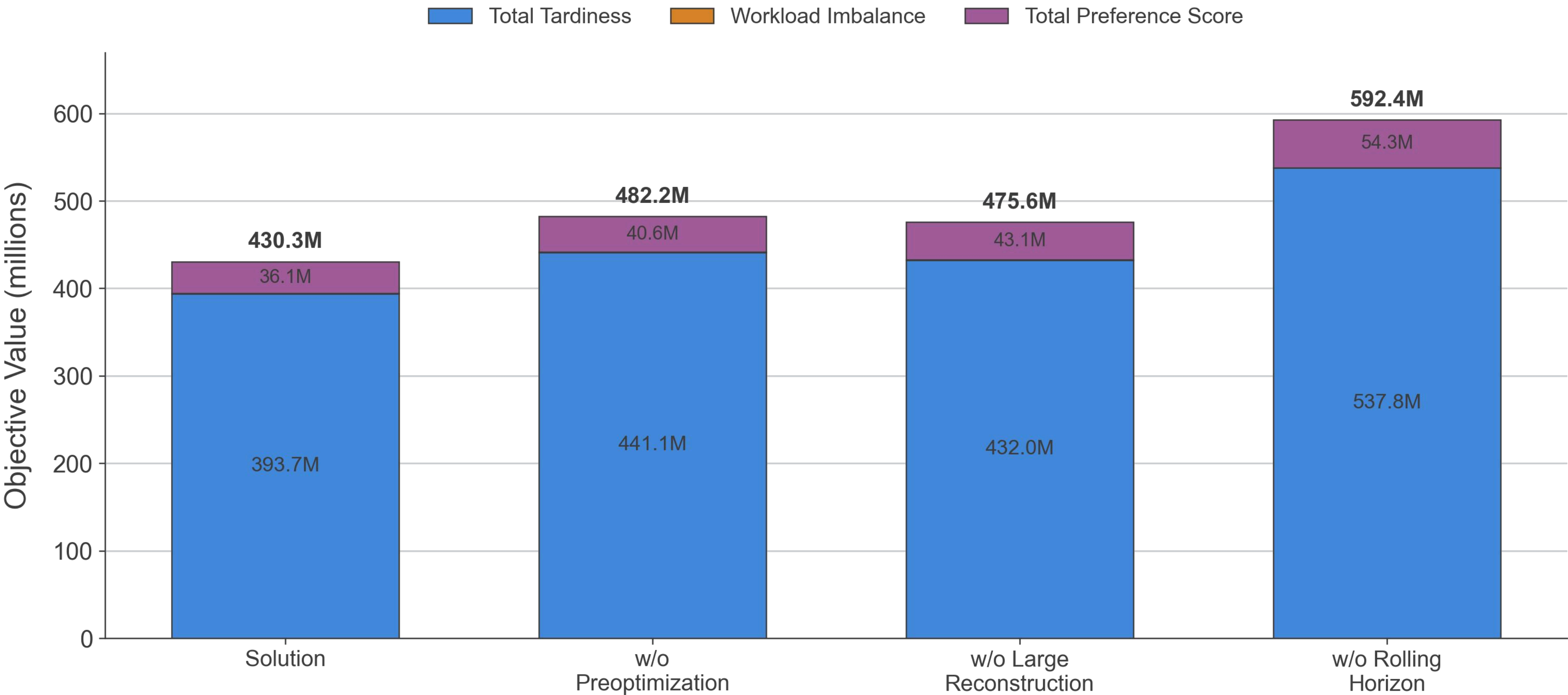
Compared Configurations

1. Solution
 - The final version submitted.
2. w/o Preoptimization
 - Abstract preoptimization was skipped, and blocks were admitted in ascending due-date order.
3. w/o Large Reconstruction
 - The probability of the large-reconstruction neighborhood was set to zero, leaving only Shift, Move, and Rotate.
4. w/o Rolling Horizon
 - The horizon size was set larger than the number of blocks, so all blocks were admitted and optimized in a single horizon.

Computation Results

Performance Comparison

All ablations increased the total objective more than 10%, removing the rolling horizon caused the largest deterioration of 37.7%.



Computation Results

Case-wise Performance Comparison

The full solution achieved the best/near-best score on most instances, while the ablations had substantial losses on several cases.

